just changing the id(s); and without any kind of collision, we can show multiple datepickers in the same page. That is the beauty of jQuery. In short, by using it, we can focus more on the functionality of the system and not just on small parts of the system. And we can write more complex code like a rich text editor and lots of other operations. But if we write jQuery code without proper guidance and proper methodology, we end up writing bad code; and sometimes that can become a nightmare for other team members to understand and modify for minor changes.

Developers often make silly mistakes during jQuery code implementation. So, based on some silly mistakes that I have encountered, here are some general guidelines that every developer should keep in mind while implementing jQuery code.

General guidelines for jQuery

1. *Try to use 'this'* instead of just using the id and class of the DOM elements. I have seen that most developers are happy with just using *$('#id')* or *$('.class')* everywhere:

```
//What developers are doing:
$('#id').click(function(){
     var oldValue = $('#id').val();
     var newValue = (oldValue * 10) / 2;
     $('#id').val(newValue);
});

//What should be done: Try to use more $(this) in your code.
$('#id').click(function(){
     $(this).val(($(this).val() * 10) / 2);
});
```

2. *Avoid conflicts:* When working with a CMS like WordPress or Magento, which might be using other JavaScript frameworks instead of jQuery, you need to work with jQuery inside that CMS or project. Then use the *noConflicts* of jQuery.

```
var $abc = jQuery.noConflict();
$abc('#id').click(function(){
//do something
});
```

3. *Take care of absent elements:* Make sure that the element on which your jQuery code is working/manipulating is not absent. If the element on which your code manipulates is dynamically added, then first find it, if that element is added on DOM.

```
$('#divId').find('#someId').length
```

This code returns 0 if there isn't an element with 'someId' found; else it will return the total number of elements that are inside 'divId'.

4. *Use proper selectors and try to use more 'find()'*, because *find* can traverse DOM faster. For example, if we want to find content of #id3…

```
//demo code snippet
<div id='#id1'>
<span id='#id2'></span>
<div class='divClass'>Here is the content.</div>
</div>

//developer generally uses
var content = $('#id1 .divClass').html();

//the better way is [This is faster in execution]
var content = $('#id1').find('div.divClass').html();
```

5. *Write functions wherever required:* Generally, developers write the same code multiple times. To avoid this, we can write functions. To write functions, let's find the block that will repeat. For example, if there is a validation of an entry for a text box and the same gets repeated for many similar text boxes, then we can write a function for the same. Given below is a simple example of a text box entry. If the value is left empty in the entry, then *function* returns 0; else, if the user has entered some value, then it should return the same value.

```
//Javascript
function doValidation(elementId){
//get value using elementId
//check and return value
}

//simple jQuery
$("input[type='text']").blur(function(){
//get value using $(this)
//check and return value
});

//best way to implement
//now you can use this function easily with click event also
$.doValidation = function(){
//get value
//check and return value
};

$("input[type='text']").blur($.doValidation);
```

6. *Object organisation:* This is another thing that each developer needs to keep in mind. If one bunch of variables is related to one task and another bunch of variables is related to another task, then get them better organised, as shown below: